

Linux Notes

Nicoll Douglas

April 9, 2026

Contents

1	The Shell	2
1.1	Shell Types	2
1.1.1	Login Shells	2
1.1.2	Interactive Shell	2
1.2	Shell Behaviour	3
1.2.1	Environment Variables	3
1.2.2	<code>exec</code>	3
1.3	Redirects and Streams	4
1.4	Here-Documents	4
1.5	Here-Strings	5
1.6	Utility Commands	5
1.6.1	Help Commands	5
1.6.2	<code>history</code>	5
2	Files	6
2.1	Permissions	6
2.1.1	Overview	6
2.1.2	Basic Permissions	6
2.1.3	SetUID Bit	7
2.1.4	SetGID Bit	8
2.1.5	Sticky Bit	8
2.1.6	Umask	9
2.2	Archiving & Compressing	9
2.2.1	<code>tar</code>	9
2.2.2	<code>zip/unzip</code>	9
2.2.3	<code>gzip/gunzip</code>	10
2.3	Links	10
2.3.1	Hard Links	10
2.3.2	Soft Links	10
2.3.3	<code>ln</code>	11

3	Processes	11
3.1	Permissions	11
3.2	Process Forking	11
3.3	Process States	12
3.4	Signals	12
4	Users	12
4.1	Switching to a User	12
4.1.1	su	12
4.1.2	sudo	12
4.2	User Management	13
4.2.1	Delegating a Superuser	13
5	Text Editing & Processing	13
5.1	Vim	13
5.1.1	netrw	13
5.1.2	Switching Modes	13
5.1.3	Navigation	14
5.1.4	Insertion & Deletion	15
5.1.5	Searching	16
5.1.6	Useful Keybinds	16

1 The Shell

1.1 Shell Types

1.1.1 Login Shells

- A login shell is a shell that starts a user session. Login shells are usually created when you:
 - Log in to a virtual console to interface with your machine (a tty device).
 - SSH into a machine.
 - Log in via a display manager.
- They typically read and execute the system-wide login shell initialization file `/etc/profile` and the per-user login initialization `~/.profile` (or `~/.bash_login` or `~/.bash_profile`).

1.1.2 Interactive Shell

- A shell where you can type in commands.
- Can be a login or non-login shell.

- Reads and executes the system-wide shell initialization file `~/etc/bash.bashrc` (Debian/Ubuntu) and the per-user initialization `~/bashrc`.
- Non-interactive shells don't have interactive features.

1.2 Shell Behaviour

1.2.1 Environment Variables

- Shell variables only exist in the current shell.
- Environment variables are part of the environment (they are exported variables).
- When you start a child process from a shell it inherits all current environment variables.
- When you run a command in a shell, the shell process forks itself and then loads a new program into the process memory as standard in Linux.
- This way the child process inherits things like environment variables, open file descriptors, cwd, etc.
- The child process will get a new process ID.
- The child process will run in the foreground (shell waits for it to finish).
- The child process can also be run in the background (shell doesn't wait for it to finish, child runs async).
- When you run a script with `source`, no child process is created, it runs in the context of the current shell (things like `cd` and changes to environment variables affect the current shell).
- When the script exits, control is handed back to the origin shell/script and it will keep running.

1.2.2 `exec`

- If you use `exec` to run a command/program, no new processes are forked, instead the shell gets replaced entirely with another program.
- When the program exits, the origin shell will no longer exist (in a script any commands after will never run).
- Much like when a new process is ran in a shell, a command ran under `exec` inherits the environment of the origin shell.
- The process ID stays the same.

Quick Usage

`exec <command>` Run a command with `exec`.

`exec > <file_path>` Permanently redirect stdout to a file for the current shell.

`exec 0< <file_path>` Permanently redirect stdin to a file for the current shell.

1.3 Redirects and Streams

Shells have three basic I/O streams:

- stdin which uses file descriptor 0.
- stdout which uses file descriptor 1.
- stderr which uses file descriptor 2.

There are also a number of operators indicated below which may be used to manipulate these streams.

Operator	Description
>	Redirect stdout to a file, overwriting it
>>	Redirect stdout to a file, appending to it
<	Redirect stdin from a file
2>	Redirect stderr to a file, overwriting it
2>>	Redirect stderr to a file, appending to it
&>	Redirect stderr and stdout to a file
<<	Here-document: redirect stdin from a block of text
<<<	Here-string: redirect stdin from a string
	Pipe: redirect stdout of one command to stdin of another
>&	Redirect output file descriptors
>&-	Close a file descriptor
<&	Redirect input file descriptors

Table 1: Redirect (and similar) operators that can be used to manipulate streams

1.4 Here-Documents

- Here-documents allow you to feed a block of text into stdin of a command.
- Requires an opening and closing delimiter (which must be on its own line).
- Variable expansion is allowed in the here-document unless you quote the delimiter (e.g "DELIMITER", or 'DELIMITER').

Syntax, where `comm` is a command:

```
comm << DELIMITER
line 1
line 2
DELIMITER
```

1.5 Here-Strings

- Similar to here-documents but allows you to feed a one-line string into stdin of a command.
- Quotes are needed if the string has spaces.
- Single quotes prevent variable expansion whereas double quotes allow it.
- You can embed new lines into the string with `\n`.

Syntax, where `comm` is a command:

```
comm <<< "my here-string"
```

1.6 Utility Commands

1.6.1 Help Commands

`which` Locate the path of a binary.

`whatis` Display the one-line man page description of a binary.

`whereis` Locate the path, source, and manual for a binary.

`man <binary>` Shows the manual page for a binary.

`help <builtin>` Shows help information for a shell built-in.

`type -a <command>` Shows all locations of an executable command and what it is (i.e binary or built-in).

1.6.2 history

- Use the `history` command to view shell command history.
- Bash has command history persisted in `~/.bash_history`.
- The max number of entries in the file is according to the `$HISTFILESIZE` environment variable.
- An in-memory history list is also kept for each shell.
- The max number of entries in the in-memory history is according to the `$HISTSIZE` environment variable.

- When a shell starts, the last `$HISTSIZE` (e.g 1000) commands are loaded from the history file into memory.
- Each time a command is ran in the shell session it is appended to the in-memory history list.
- When the shell closes, any new items in the history are appended to the history file.

Quick Usage

`history <n>` Shows the last `n` commands from your in-memory history.

`history !<n>` Repeat command number `n` in your history.

Reverse Incremental Search

- `ctrl` + `r` can be used in the terminal to perform a reverse incremental search for a command or term in your history, finding the first match.
- `ctrl` + `r` can be pressed again to find the next match.

2 Files

2.1 Permissions

2.1.1 Overview

The three basic file permissions are read, write, and execute which are each applied to the user categories associated with the file: the owner, users in the file's assigned group, and all other users (usually lettered `u`, `g`, and `o`).

Permissions are stored as a 16-bit integer in a file or directory's metadata where certain values in bit ranges or individual bits correspond to a certain permission or flag. The pattern in bits 12–15 indicate the file type (e.g regular file, directory, symlink, etc), bits 9–11 contain flags for special bits (setUID, setGID and the sticky bit), 6–8 contain the owner permissions, 3–5 the group permissions, and 0–2 the other user permissions.

2.1.2 Basic Permissions

The three basic permissions are read, write, and execute which are stored as 3 bits for each of the different user categories (usually letter `r`, `w`, and `x`).

Read Stored as the order 2 bit (contributes a value of 4 to a set of permission bits).

- **For files:** Allows you to view the contents of a file (e.g `cat file.txt`).
- **For directories:** Allows you to view the contents of a directory (e.g `ls -ld`).

Write Stored as the order 1 bit (contributes a value of 2 to a set of permission bits).

- **For files:** Allows you to modify the contents of a file (e.g `echo "modified" >> file.txt`).
- **For directories:** Allows you to create, delete, or rename files in a directory (e.g `rm file.txt`). You also need execute permissions on the directory in order to write.

Execute Stored as the order 0 bit (contributes a value of 1 to a set of permission bits).

- **For files:** Allows you to execute the contents of the file as a program/script.
- **For directories:** Allows you enter a directory and access files essentially acting as a gatekeeper to anything inside (e.g gatekeeps things actions like `cd dir` or `cat dir/file.txt`).

2.1.3 SetUID Bit

- The setUID bit is a "permission" used on **files**.
- It is set using the following command:

```
chmod u+s <filename>
```

- When a file is executed that has the setUID bit is set, the EUID of the process is set to the file owner's UID and **not** the UID of the user executing the file.
- This allows a user to execute the file and give the process the owner's permissions.

Display of the setUID bit in file permissions:

-rwsrw-r- Uppercase s means the file owner *can't* execute but the setUID bit is set.

-rwsrw-r- Lowercase s means the file owner *can* execute and the setUID bit is set.

2.1.4 SetGID Bit

- The setGID bit is another "permission" used on **files and directories**.
- It is set using the following command:

```
chmod g+s <file_or_dir>
```

- For files:
 - When the file is executed, the EGID of the process is set to the file's group's GID and **not** the primary group GID of the user executing the file.
 - This allows users to execute with the file's group's permissions.
- For directories:
 - Any new files or directories created **only** in that directory (1 level) will inherit the directory's group.

Display of the setGID bit in file permissions:

-rw-rwSr- Uppercase s means the file or directory group *doesn't* have execute but the setGID bit is set.

-rw-rwsr- Lowercase s means the file or directory group *does* have execute and the setGID bit is set.

2.1.5 Sticky Bit

- The sticky bit is used on **directories**.
- It is set using the following command:

```
chmod +t <directory>
```

- In order to delete or rename files in a directory, you need write and execute permissions on the directory.
- The sticky bit adds an extra restriction: **only the owner of the file or directory can delete or rename a file**.
- This means users in the directory group and other users can't delete or rename files even if they have write and execute permissions.

Display of the sticky bit in file permissions:

-rw-rw-r-T Uppercase t means other users *don't* have directory execute but the sticky bit is set.

-rw-rw-r-t Lowercase t means other users *do* have directory execute and the sticky bit is set.

2.1.6 Umask

- The umask is a per-process configuration that dictates what permissions new files and directories created by that process will have.
- New files are *usually never* created with executable permissions.
- New directories are *usually* created with executable permissions.
- The umask is represented in octal, so the permission bits are masked (using bitwise AND) with the umask to remove the bits set in the umask.
- Example, `umask 022`:
 - Files usually don't start with execute bits so permissions start at 666.
 - Then, the umask masks off `gx` and `ox` permission bits (set in the umask), leaving the permission bits as octal 644.
- 022 is a common umask, for my machine (Debian 13) it is 002.
- Umask is per-process so when a process starts it takes on the parent process umask (due to process forking but is usually the default).

2.2 Archiving & Compressing

2.2.1 tar

The `tar` command is used to create and extract tar archives (tarballs).

Quick Usage

`tar -cvf <tar_filename> <target>` Create a tar archive.

`tar -cvzf <targz_filename> <target>` Create a compressed tar archive using gzip compression.

`tar -xvf <tar_filename> -C <target>` Extract a tar archive.

`tar -xvzf <targz_filename> -C <target>` Extract a gzip-compressed tar archive.

2.2.2 zip/unzip

Quick Usage

`zip <zip_filename> <file1> <file2> ...` Zip 1 or more files.

`zip -r <zip_filename> <dir>` Zip a directory.

`unzip <zip_filename>` Extract all files a zip file into the current directory.

`unzip <zip_filename> -d <directory>` Extract all files in a zip file into a directory (creating it if it doesn't exist).

2.2.3 gzip/gunzip

The `gzip` command compresses a single file using gzip compression. For multiple files use `tar`.

Quick Usage

`gzip <filename>` Gzip a file and delete the original.

`gzip -k <filename>` Gzip a file and keep the original.

`gunzip <gz_filename>` Gunzip a file.

2.3 Links

2.3.1 Hard Links

- Each file on a file system has an inode that stores metadata and pointers to the file's data blocks.
- A hard link is just another filename that points to the same inode.
- The number after a file's permissions is the number of hard links in `ls -l` output.
- You can check the inode number a file with `ls -li`.
- Hard links cannot cross filesystems (including mounted filesystems).
- Deleting a hard link does not delete a file.
- A file is only deleted when the hard link count reaches 0.
- Hard links share the same metadata and file content, the only thing that differs is the filename which is just a label.

2.3.2 Soft Links

- A soft link (a.k.a a symlink) is a separate file that points to another file (its file path).
- If the original file is deleted, the link becomes a "dangling link".
- Directories can also be symlinked.
- Soft links don't receive their source's file content directly, rather the path is followed to the content when reading.
- Soft links follow the path they point to so if the source path is relative, moving the symlink file may break it.
- Soft links that point to absolute paths are more stable and can be moved around.

2.3.3 ln

The `ln` command is used for creating hard and soft links. The source path can be relative or absolute. If relative, the source is relative to the location of the link file.

Quick Usage

`ln <source_path> <link_path>` Create a hard link to the inode of the file at `<source_path>` at `<destination_path>`.

`ln -s <source_path> <destination_path>` Create a soft link to `<source_path>` at `<destination_path>`.

3 Processes

3.1 Permissions

Each process when started has the following user and group ID fields attached to it that dictate its permissions:

RUID (Real User ID) Set to the ID of the user that started the process.

EUID (Effective UID) Set to whoever is given effective control of the process (usually UID but this may change with the `setUID` bit set on an executable file or at will by the process)

Saved Set-UID Set to whatever the initial EUID was. This is done in order for the process to remember what the EUID initially was so it can modify the EUID at will and reinstate the initial if necessary.

RGID (Real Group ID) RUID but set to the group ID of the user's primary group.

EGID (Effective GID) EUID but set to a group ID and can change based on the `setGID` bit.

Saved Set-GID The group ID equivalent of the saved set-UID.

3.2 Process Forking

- Processes are almost always created by a parent process.
- The parent process calls the `fork()` system call, creating a near-exact copy of the parent process.
- When the copy is made, the child process usually calls the `exec()` system call which replaces the child's program with a new program intended to run.

- The very first process created after your system boots is called `init` and is usually `systemd` in modern distros.
- Process forking means that every process corresponds to a node in a process tree (can be seen with `ps tree`) with `init` being the root node.

3.3 Process States

Processes can be in one of the following states:

Running (R) The process is actively executing its program or waiting in the run queue.

Sleeping (S) The process is waiting for an event (e.g user input, a file read).

Stopped (T) The process has been paused (usually by a signal like `ctrl + z`).

Zombie (Z) The process has finished but its parent hasn't acknowledged this yet.

3.4 Signals

4 Users

4.1 Switching to a User

4.1.1 `su`

- Used to give a full login shell as another user.
- Requires the user's password.
- You can also run individual commands as another user using `su`.
- `su` requires the *target user's* password so `sudo` might be preferred for automation tasks.

Quick Usage

`su - <user>` Run an interactive login shell for `<user>` (or `root` if left blank).

4.1.2 `sudo`

- A login shell can be granted with `sudo` as with `su`.
- Can also be used to run individual commands as another user.
- `sudo` requires *your* password.
- On a Linux system, the `root` user has the highest privileges.
- Users can be added to the `sudo` group to mark them as a *superuser* which lets you run commands with administrator privileges.

Quick Usage

`sudo -u <user> <command>` Run `<command>` as `<user>` (root if user not specified).

`sudo -u <user> -i` Run an interactive login shell for `<user>` (root if left blank).

4.2 User Management

4.2.1 Delegating a Superuser

`sudo usermod -aG sudo <username>` Creates a superuser by adding them to the `sudo` group.

`sudo deluser <username> sudo` Removes a superuser by removing them from the `sudo` group.

5 Text Editing & Processing

5.1 Vim

5.1.1 netrw

`netrw` is the built-in file explorer in Vim. See below the common `netrw` keybinds:

Keybind	What it Does
<code>%</code>	Create a new file
<code>d</code>	Create a new directory
<code>-</code>	Go up one directory
<code>u</code>	Return to the previous directory
<code>r</code>	Refresh the directory listing
<code>↑ + r</code>	Rename a file
<code>↑ + d</code>	Delete a file
<code>leader + e</code>	Open <code>netrw</code> .
<code>leader + q</code>	Close <code>netrw</code> .

Table 2: Common keybinds in `netrw`

5.1.2 Switching Modes

Keybinds See below the common keybinds for switching modes in Vim:

Keybind	What it Does
<code>j</code> + <code>j</code>	Switch from insert to normal mode
<code>v</code>	Switch from insert to visual mode
<code>leader</code> + <code>leader</code>	Switch from visual to normal mode
<code>↑</code> + <code>v</code>	Select a line in visual mode
<code>ctrl</code> + <code>v</code>	Enter visual mode and select a rectangular block of text
<code>a</code>	Enter insert mode and start inserting <i>after</i> the caret
<code>i</code>	Enter insert mode and start inserting <i>before</i> the caret
<code>↑</code> + <code>a</code>	Enter insert mode and start inserting at the <i>end</i> of the line
<code>↑</code> + <code>i</code>	Enter insert mode and start inserting at the <i>start</i> of a line
<code>o</code>	Open a new line <i>below</i> the current and enter insert mode
<code>↑</code> + <code>o</code>	Open a new line <i>above</i> the current and enter insert mode

Table 3: Common keybinds used for switching modes in Vim

5.1.3 Navigation

Keybinds See below the common navigation keybinds:

Keybind	What it Does
h	Move left
j	Move down
k	Move up
l	Move right
0	Go to the <i>first character</i> of the line
\$	Go to the <i>last character</i> of the line
^	Go to the <i>first non-blank character</i> of the line
g + g	Go to the <i>first line</i> of the file
↑ + g	Go to the <i>last line</i> of the file
w	Go to the start of the next word
b	Go to the start of the previous word
e	Go to the end of the next word
f + <char>	Find the next occurrence of <char> and jump
↑ + f + <char>	Find the previous occurrence of <char> and jump
;	Repeat the last f -based keybind
,	Repeat the last f -based keybind in the reverse direction
%	Go to the matching bracket of the current
↑ + m	Go to the line in the middle of the screen
↑ + h	Go to the line at the top of the screen
↑ + l	Go to the line at the bottom of the screen

Table 4: Common keybinds used for navigation in Vim

Commands See below the common navigation commands:

Command	What it Does
:<number>	Move to the line numbered <number>
:\$	Move to the last line in the file
:-<number>	Move <number> lines <i>up</i>
:+<number>	Move <number> lines <i>down</i>

Table 5: Common commands used for navigation in Vim

5.1.4 Insertion & Deletion

Keybinds See below the common keybinds used for inserting and deleting content in Vim:

Keybind	What it Does
 + 	Cut a word starting at the caret
 + 	Delete a line
	Undo the last action
 + 	Redo the last action
	Paste the last cut item (e.g line or character) after the caret or after the current line
 + 	Past the last cut item before the caret or before the current line
 + 	Copy a word
 + 	Copy a line
 +  + 	Cut the current word
 +  + 	Copy the current word

Table 6: Common keybinds used for inserting and deleting content in Vim

5.1.5 Searching

Keybinds See below the common keybinds used for searching in Vim:

Keybind	What it Does
	Go to the next match
 + 	Go to the previous match

Table 7: Common keybinds used for searching in Vim

Commands See below the common commands used for searching in Vim:

Command	What it Does
/	Open a search query
?	Open a search query in the reverse direction

Table 8: Common commands used for searching in Vim

5.1.6 Useful Keybinds

See below other useful keybinds in Vim:

Keybind	What it Does
 + 	Scroll the current line to the middle of the screen
	Repeat the last change made in normal mode
 +  + 	Save and quit vim

Table 9: Common commands used for searching in Vim